



Esercizio: ZombieChat – la chat dell’apocalisse

Durante un’apocalisse zombie, un gruppo di sopravvissuti comunica tramite una chat condivisa chiamata **ZombieChat**.

Ogni sopravvissuto (thread) invia messaggi per coordinarsi (“Ho trovato cibo”, “Zombie a nord”, ecc.).

Quando più thread accedono contemporaneamente alla stessa struttura dati, possono verificarsi anomalie: messaggi che “spariscono” e contatori incoerenti.

Obiettivo

Simulare una chat condivisa tra più thread e osservare cosa succede **con e senza sincronizzazione**, sia su una lista condivisa sia su un contatore intero.

Struttura del sistema

La classe `ZombieChat` contiene:

- una lista di messaggi (`List<String>`);
- un contatore intero `int contatoreMessaggi`;
- due metodi per inviare messaggi:
 - `inviaMessaggio(String msg)` → senza sincronizzazione;
 - `inviaMessaggioSync(String msg)` → con sincronizzazione.

Ogni invio di messaggio deve:

- aggiungere il messaggio alla lista;
- incrementare il contatore (`contatoreMessaggi++`).

Scenario

- **5 thread (sopravvissuti)**
- ogni thread invia **10 messaggi**
- nomi dei thread:
 - `ZombieMario`
 - `ZombieLuisa`
 - `ZombieGino`
 - `ZombiePina`
 - `ZombieBruno`

Totale atteso:

- lista: **50 messaggi**
- contatore: **50**

1. Classe `ZombieChat`

Implementare:

- `inviaMessaggio(String msg)` (non sincronizzato):
 - aggiunge il messaggio alla lista;
 - incrementa `contatoreMessaggi`;
 - `inviaMessaggioSync(String msg)` (sincronizzato):
 - stessa logica, ma protetta con `synchronized`;
 - `getNumeroMessaggi()` → dimensione lista;
 - `getContatoreMessaggi()` → valore del contatore;
 - `getMessaggi()` → lista completa.
-

2. Classe `SopravvissutoThread`

Estende `Thread`.

Il costruttore deve ricevere:

- riferimento alla `ZombieChat` condivisa;
- nome del thread;
- lista di messaggi possibili (da cui scegliere in modo casuale);
- un parametro booleano (es. `usaSync`) che decide quale metodo usare.

Nel metodo `run()`:

- ogni thread invia **10 messaggi**;
- ogni messaggio è scelto casualmente dalla lista;
- in base al parametro:
 - se `usaSync == true` → usa `inviaMessaggioSync()`
 - altrimenti → usa `inviaMessaggio()`

Formato messaggi:

```
ZombieMario: ho visto uno zombie  
ZombieMario: zona sicura  
ZombieMario: serve aiuto
```

3. Classe `ZombieChatMain`

- crea un'unica istanza di `ZombieChat` (oggetto condiviso / monitor);
- prepara una lista di messaggi possibili (es. 4–5 frasi);
- crea i 5 thread con i nomi indicati;
- passa a ogni thread:
 - la chat
 - la lista messaggi
 - il parametro `usaSync` impostato a falso e poi a vero (vedete voi se fare due main o cambiarlo a mano)
- avvia tutti i thread;
- attende la fine con `join()` ;

Alla fine stampa:

```
Messaggi nella lista: XXX  
Valore contatore: YYY  
  
Contenuto della chat:  
[messaggio1, messaggio2, messaggio3, ...]
```

4. Test richiesti

Versione senza sincronizzazione

- creare i thread con `usaSync = false`

Possibile risultato:

```
Messaggi nella lista: 47  
Valore contatore: 43
```

Versione con sincronizzazione

- creare i thread con `usaSync = true`

Risultato atteso:

```
Messaggi nella lista: 50
Valore contatore: 50
```

Se il problema non è evidente:

- aumentare il numero di messaggi (es. 100);
- aumentare i thread;
- inserire un breve `sleep`.

Domande di riflessione

1. Perché lista e contatore non assumono sempre valore 50 senza sincronizzazione?
2. Perché i due valori possono essere diversi tra loro?
3. Cos'è una race condition?
4. Perché `ArrayList` non è thread-safe?
5. Perché anche `contatoreMessaggi++` non è sicuro?
6. Quali operazioni compongono l'incremento?
7. In che modo `synchronized` risolve il problema?
8. Che cos'è il lock implicito?

Consegna

- `ZombieChat.java`
- `SopravvissutoThread.java`
- `ZombieChatMain.java`
- `README.txt` con:
 - risposte teoriche;
 - risultati di almeno 3 esecuzioni per entrambe le versioni.

Nota concettuale

L'istruzione:

```
contatoreMessaggi++;
```

non è atomica, ma composta da più operazioni:

```
lettura → incremento → scrittura
```

Se due thread eseguono queste operazioni contemporaneamente, uno degli incrementi può andare perso.